

监督学习 - 双相障碍检测

1. 实验介绍

1.1 实验背景

双相障碍属于心境障碍的一种疾病，英文名称为 Bipolar Disorder (BD)，别名为 Bipolar Affective Disorder，表示既有躁狂发作又有抑郁发作的一类疾病。

目前病因未明，主要是生物、心理与社会环境诸多方面因素参与其发病过程。

当前研究发现，在双相障碍发生过程中遗传因素、环境或应激因素之间的交互作用、以及交互作用的出现时间点等都产生重要的影响；临床表现按照发作特点可以分为抑郁发作、躁狂发作或混合发作。

双相障碍检测，即通过医学检测数据预测病人是否双相障碍，或双相障碍治疗是否有效。

医学数据包括医学影像数据与肠道数据。

由于缺少医学样本且特征过多，因此选取合适的特征对双模态特征进行整合并训练合适的分类器进行模型预测具有较强的现实需求与医学意义。

本实验需要大家完成少样本、多特征下的监督学习。

1.2 实验要求

- 实现双模态特征选择与提取整合。
- 选择并训练机器学习模型进行准确分类。
- 分析不同超参数以及特征选择方法对模型的结果影响。

1.3 实验环境

可以使用 Numpy 库进行相关数值运算，使用 sklearn 库进行特征选择和训练机器学习模型等。

1.4 参考资料

Numpy: <https://www.numpy.org/>

Scikit-learn: <https://scikit-learn.org/>

2. 实验内容

2.1 实验准备

导入实验所需的库

```
# 医疗数据集存放在左侧栏中的 `DataSet.xlsx` 中，共包括 39 个样本和 3 张表，表 `Feature1` 为医学影像特征，表 `Feature2` 为肠道特征，表 `label` 为样本类标。
```

```

# 导入相关库
import warnings
import itertools
import numpy as np
import pandas as pd
import seaborn as sns
from time import time
from minepy import MINE
from sklearn import svm
from sklearn import tree
import matplotlib.pyplot as plt
from sklearn import naive_bayes
from scipy.stats import pearsonr
from sklearn.manifold import TSNE
from IPython.display import display
from datetime import datetime as dt
from sklearn.externals import joblib
from sklearn.decomposition import PCA
from sklearn.metrics import fbeta_score
from sklearn.metrics import make_scorer
from sklearn.metrics import recall_score
from sklearn.model_selection import KFold
from sklearn.feature_selection import chi2
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import ShuffleSplit
from sklearn.model_selection import GridSearchCV
from sklearn.feature_selection import SelectKBest
from sklearn.model_selection import learning_curve
from sklearn.model_selection import train_test_split

warnings.filterwarnings('ignore')
%matplotlib inline

```

2.2 准备数据

数据预处理 是一种数据挖掘技术，它是指把原始数据转换成可以理解的格式。在这个过程中一般有数据清洗、数据变换、数据组织、数据降维和格式化等操作。

对于本数据集，没有无效或丢失的条目；然而需要进行特征的筛选和整合。

我们可以针对某一些特征存在的特性进行一定的调整。

这些预处理可以极大地帮助我们提升机器学习算法模型的性能和预测能力。

归一化数字特征

对数值特征施加一些形式的缩放，可以减少量纲对数据的影响。

对数据分析发现，`Feature2` 中的特征值存在较大差异，比如第 0 维和第 374 维；大家可以试试观察其它列特征是否有这种现象？

数据归一化的作用：

- 1) 把数据变成 (0,1) 或者 (-1,1) 之间的小数。主要是为了数据处理方便提出来的，把数据映射到 0 ~ 1 范围之内处理，更加便捷快速。
- 2) 把有量纲表达式变成无量纲表达式，便于不同单位或量级的指标能够进行比较和加权。

注意：一旦使用了缩放，观察数据的原始形式不再具有它本来的意义了。

我们将使用 `sklearn.preprocessing.MinMaxScaler` 来完成这个任务。

```
def processing_data(data_path):  
    """  
    数据处理  
    :param data_path: 数据集路径  
    :return: feature1,feature2,label: 处理后的特征数据、标签数据  
    """  
  
    #导入医疗数据  
    data_xls = pd.ExcelFile(data_path)  
    data={}  
  
    #查看数据名称与大小  
    for name in data_xls.sheet_names:  
        df = data_xls.parse(sheet_name=name,header=None)  
        data[name] = df  
  
    #获取 特征1 特征2 类标  
    feature1_raw = data['Feature1']  
    feature2_raw = data['Feature2']  
    label = data['label']  
  
    # 初始化一个 scaler, 并将它施加到特征上  
    scaler = MinMaxScaler()  
    feature1 = pd.DataFrame(scaler.fit_transform(feature1_raw))  
    feature2 = pd.DataFrame(scaler.fit_transform(feature2_raw))  
  
    return feature1,feature2,label
```

2.3 评价模型性能

我们的研究目的，是通过医学检测数据预测病人是否双相障碍，或双相障碍治疗是否有效。

因此，对于准确预测病人是否双相障碍，或双相障碍治疗是否有效是问题的关键。

这样看起来使用**准确率**作为评价模型的标准是合适的。

我们将算法预测结果分为四种情况：

		预测值	
		Positive	Negative
实际值	Positive	True Positive (TP)	False Negative (FN)
	Negative	False Positive (FP)	True Negative (TN)

准确率 (Accuracy) 是指分类正确的样本占总样本个数的比例

$$accuracy = \frac{\text{预测正确的样本数}}{\text{总样本数}} = \frac{TP+TN}{TP+TN+FP+FN}$$

但是，把双相障碍的病人预测为正常人，或者把治疗无效预测为有效是存在极大的医学隐患的。

我们期望的模型具有能够 **查全** 所有双相障碍病人或者双相治疗有效法人病例与模型的准确预测**同样重要**。

因此，我们使用 **查全率 (Recall)** 作为评价模型的另一标准。

查准率 (Precision) **在算法预测都为正类 (Positive) 样本中，实际是正类 (Positive) 样本的比例**

$$precision = \frac{TP}{TP+FP}$$

查全率 (Recall) 在实际值是正类 (Positive) 的样本中，算法预测是正类样本的比例

$$recall = \frac{TP}{TP+FN}$$

我们使用 **F-beta score** 作为评价指标，这样能够同时考虑查准率和查全率：

$$F_{\beta} = (1 + \beta^2) \cdot \frac{precision \cdot recall}{(\beta^2 \cdot precision) + recall}$$

当 $\beta = 1$ 时，就是我们常听说的 **F1 值 (F1 score)**

当 $\beta = 0.5$ 的时候更多的强调查准率，这叫做 **F_{0.5} score** (或者为了简单叫做 F-score)

2.4 特征选取

使用监督学习算法的一个重要的任务是决定哪些数据特征能够提供最强的预测能力。

专注于少量的有效特征和标签之间的关系，我们能够更加简单具体地理解标签与特征之间的关系，这在很多情况下都是十分有用的。

可以看到：医疗数据中的样本和特征数量存在着极大的不平衡，其中医疗影像数据共 6670 维，肠道数据共 377 维，而样本仅有 39 个。

因此，为了训练预测模型，特征的筛选和组合以及机器学习模型的选择优化极其重要。

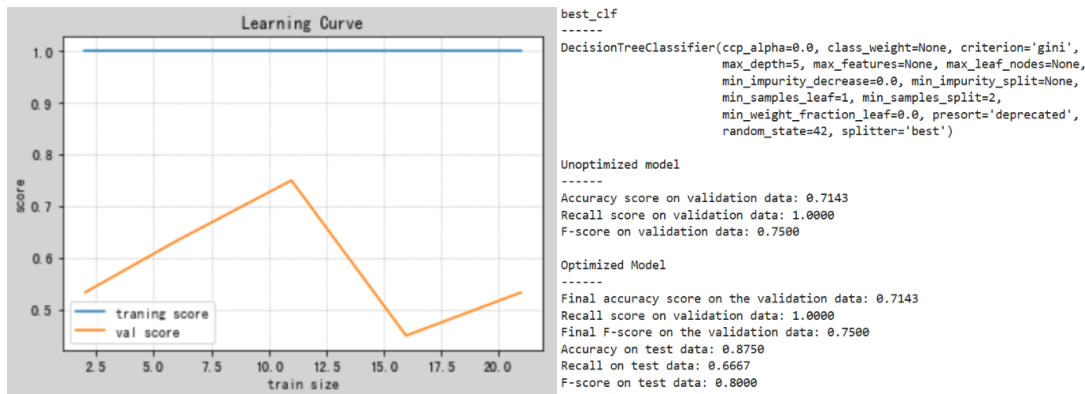
同时，在这个项目的情境下选择一小部分特征，也具有很大的医学意义。

皮尔逊相关系数

```
def feature_select(feature1, feature2, label):  
    """  
    特征选择  
    :param feature1, feature2, label: 数据处理后的输入特征数据，标签数据  
    :return: new_features, label: 特征选择后的特征数据、标签数据  
    """  
    new_features = None  
    # 皮尔逊相关系数  
    # 整合特征  
    features = pd.concat([feature1, feature2], axis=1)  
  
    # 统计特征值和label的皮尔孙相关系数 进行排序筛选特征  
    select_feature_number = 12  
    select_features = selectKBest(lambda x, Y: tuple(map(tuple,  
np.array(list(map(lambda x: pearsonr(x, Y), X.T))).T)),  
  
    k=select_feature_number).fit(features, np.array(label).flatten()).get_support(in  
dices=True)  
  
    # 查看提取的特征序号  
    print("查看提取的特征序号:", select_features)  
  
    # 特征选择
```

```
new_features = features[features.columns.values[select_features]]
return new_features, label
```

先按照所给示例进行模型训练，以皮尔逊相关系数为例，进行特征选择并得到新特征数据，得到结果如下图所示



可以发现效果不是很好，因此对训练模型进行修改

双模态特征选择和融合

以上特征选择都是在将医疗影像数据和肠道数据直接拼接后进行的。但是事实上，双模态特征各自具有不同的分布和医学意义，因此，分别对各特征进行筛选，再按照相关算法进行特征的融合是比较合理的方法。

```
def feature_select(feature1, feature2, label):
    # 统计特征值和label的皮尔逊相关系数 对两类特征分别进行排序筛选特征
    select_feature_number = 5
    select_feature1 = SelectKBest(lambda X, Y:
tuple(map(tuple,np.array(list(map(lambda x:pearsonr(x, Y), X.T))))),
                                k=select_feature_number
                                ).fit(feature1,
np.array(label).flatten()).get_support(indices=True)

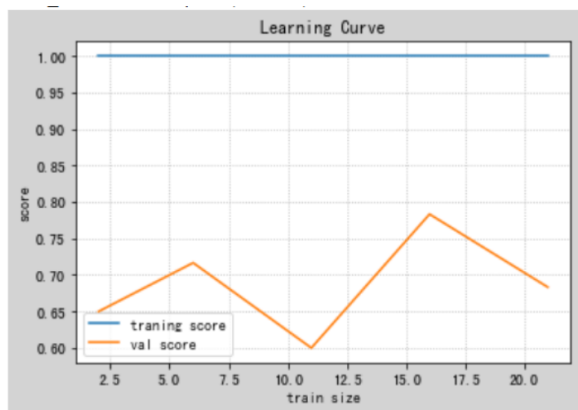
    select_feature2 = SelectKBest(lambda X, Y:
tuple(map(tuple,np.array(list(map(lambda x:pearsonr(x, Y), X.T))))),
                                k=select_feature_number
                                ).fit(feature2,
np.array(label).flatten()).get_support(indices=True)

    # 查看排序后特征
    print("select feature1 name:", select_feature1)
    print("select feature2 name:", select_feature2)

    # 双模态特征选择并融合
    new_features =
pd.concat([feature1[feature1.columns.values[select_feature1]],

feature2[feature2.columns.values[select_feature2]]],axis=1)
    print("new_features shape:",new_features.shape)
    return new_features, label
```

结果如下图所示



```
best_clf
-----
DecisionTreeClassifier(ccp_alpha=0.0, class_weight=None, criterion='gini',
                        max_depth=5, max_features=None, max_leaf_nodes=None,
                        min_impurity_decrease=0.0, min_impurity_split=None,
                        min_samples_leaf=1, min_samples_split=2,
                        min_weight_fraction_leaf=0.0, presort='deprecated',
                        random_state=42, splitter='best')

Unoptimized model
-----
Accuracy score on validation data: 0.7143
Recall score on validation data: 0.6667
F-score on validation data: 0.6667

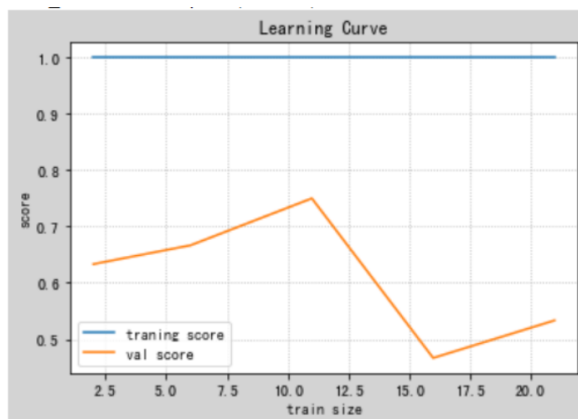
Optimized Model
-----
Final accuracy score on the validation data: 0.7143
Recall score on validation data: 0.6667
Final F-score on the validation data: 0.6667
Accuracy on test data: 0.8750
Recall on test data: 0.6667
F-score on test data: 0.8000
```

可以发现有所改变，但效果依然不是很好，因此考虑进行参数调整，修改 `select_feature_number`，修改如下

```
select_feature_number1 = 25
select_feature_number2 = 10
select_feature1 = SelectKBest(lambda X, Y:
tuple(map(tuple,np.array(list(map(lambda x:pearsonr(x, Y), X.T))))),
                             k=select_feature_number1
                             ).fit(feature1,
np.array(label).flatten()).get_support(indices=True)

select_feature2 = SelectKBest(lambda X, Y:
tuple(map(tuple,np.array(list(map(lambda x:pearsonr(x, Y), X.T))))),
                             k=select_feature_number2
                             ).fit(feature2,
np.array(label).flatten()).get_support(indices=True)
```

现在的结果如下图所示



```
best_clf
-----
DecisionTreeClassifier(ccp_alpha=0.0, class_weight=None, criterion='gini',
                        max_depth=5, max_features=None, max_leaf_nodes=None,
                        min_impurity_decrease=0.0, min_impurity_split=None,
                        min_samples_leaf=1, min_samples_split=2,
                        min_weight_fraction_leaf=0.0, presort='deprecated',
                        random_state=42, splitter='best')

Unoptimized model
-----
Accuracy score on validation data: 0.7143
Recall score on validation data: 0.6667
F-score on validation data: 0.6667

Optimized Model
-----
Final accuracy score on the validation data: 0.7143
Recall score on validation data: 0.6667
Final F-score on the validation data: 0.6667
Accuracy on test data: 0.8750
Recall on test data: 0.6667
F-score on test data: 0.8000
```

效果并没有改善，猜测可能是降维方面需要完善，因此采用PCA进行降维处理，经过不断的调参，修改数据切分参数与超参数

```

# ----- 实现数据切分部分代码 -----
# 将 features 和 label 数据切分成训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(features, label,
test_size=0.2, random_state=10, stratify=label)

# 将 X_train 和 y_train 进一步切分为训练集和验证集
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train,
test_size=0.2, random_state=5, stratify=y_train)
# -----

# 创建调节的参数列表
parameters = {'max_depth': range(3,10),
              'min_samples_split': range(5,10)}

```

最终的代码如下所示

```

def feature_select(feature1, feature2, label):
    """
    特征选择
    :param feature1, feature2, label: 数据处理后的输入特征数据, 标签数据
    :return: new_features, label: 特征选择后的特征数据、标签数据
    """
    new_features= None
    # 统计特征值和label的皮尔孙相关系数 对两类特征分别进行排序筛选特征
    select_feature_number1 = 25
    select_feature_number2 = 10
    select_feature1 = SelectKBest(lambda X, Y:
tuple(map(tuple,np.array(list(map(lambda x:pearsonr(x, Y), X.T))).T)),
                                k=select_feature_number1
                                ).fit(feature1,
np.array(label).flatten()).get_support(indices=True)

    select_feature2 = SelectKBest(lambda X, Y:
tuple(map(tuple,np.array(list(map(lambda x:pearsonr(x, Y), X.T))).T)),
                                k=select_feature_number2
                                ).fit(feature2,
np.array(label).flatten()).get_support(indices=True)

    # 查看排序后特征
    print("select feature1 name:", select_feature1)
    print("select feature2 name:", select_feature2)

    #相关系数筛选出的特征
    s1_feature1=feature1[feature1.columns.values[select_feature1]]
    s1_feature2=feature2[feature2.columns.values[select_feature2]]

    # ----- PCA降维部分代码 -----
    # 选择降维维度
    pca1 = PCA(n_components=2)
    pca2 = PCA(n_components=1)
    feature_pca1 = pca1.fit_transform(s1_feature1)
    feature_pca2 = pca2.fit_transform(s1_feature2)

    # 获取每个主成分的方差变化信息
    variance_explained1 = pca1.explained_variance_ratio_
    variance_explained2 = pca2.explained_variance_ratio_

```

```

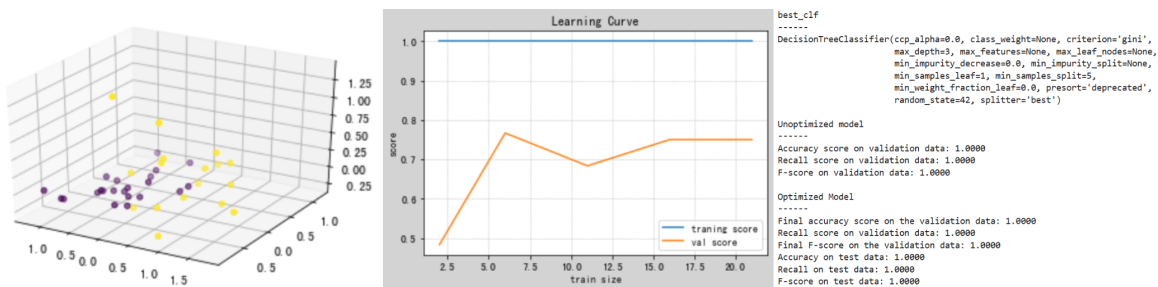
print("主成分1的方差变化信息: ", variance_explained1)
print("主成分2的方差变化信息: ", variance_explained2)
from mpl_toolkits.mplot3d import Axes3D

# 可视化标签中不能出现负值
pca_label = np.array(label).flatten()
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(feature_pca1[:, 0], feature_pca1[:, 1], feature_pca2[:, 0], c=pca_label)
plt.show()
# 对 feature1 和 feature2 进行整合
feature_pca1_df = pd.DataFrame(feature_pca1)
feature_pca2_df = pd.DataFrame(feature_pca2)

new_features = pd.concat([feature_pca1_df, feature_pca2_df], axis=1)
# -----
# 返回筛选后的数据
return new_features, label

```

结果如下所示



准确率为 1.0, 召回率为 1.0, F-score 为 1.0

3. 测试结果

测试详情

测试点	状态	时长	结果
测试结果	✓	3s	测试成功, 在10个测试样本中, 准确率为 1.0, 召回率为 1.0, F-score 为 1.0

确定

4. 实验心得

对于高维的数据集, 直接用一种特征选择方法将其筛选过于简单粗暴, 模型没有获取足够的信息, 很难有好的预测结果。因此我开始尝试逐步筛选——先筛掉相关性或者重要性较弱的特征, 保留 50 个左右, 再用 PCA 分别提取主成分并拼接, 从而将特征降至 3 维, 在经过中间的多种尝试后, 我才找到了一组表现良好的特征。

除了特征工程, 我还做了一些其他小的改动, 如更换模型、改变数据集切分比例、改变 random_state 等等, 但是这些对预测效果的影响都不是很大, 我就没有做进一步的探究了。

在本次实验中，我不仅感受到了特征工程的魅力，也意识到了训练监督学习模型这一过程是需要技巧与耐心的，运用合适的方法，再沉下心来慢慢调整，总能调出优秀的模型的

5. 附件

5.1 训练机器学习模型

```
def processing_data(data_path):  
    """  
    数据处理  
    :param data_path: 数据集路径  
    :return: feature1,feature2,label:处理后的特征数据、标签数据  
    """  
    feature1,feature2,label = None, None, None  
    # ----- 实现数据处理部分代码 -----  
  
    #导入医疗数据  
    data_xls = pd.ExcelFile(data_path)  
    data={}  
  
    #查看数据名称与大小  
    for name in data_xls.sheet_names:  
        df = data_xls.parse(sheet_name=name,header=None)  
        data[name] = df  
  
    #获取 特征1 特征2 类标  
    feature1_raw = data['Feature1']  
    feature2_raw = data['Feature2']  
    label = data['label']  
  
    # 初始化一个 scaler，并将它施加到特征上  
    scaler = MinMaxScaler()  
    feature1 = pd.DataFrame(scaler.fit_transform(feature1_raw))  
    feature2 = pd.DataFrame(scaler.fit_transform(feature2_raw))  
    # -----  
  
    return feature1,feature2,label  
  
def feature_select(feature1, feature2, label):  
    """  
    特征选择  
    :param feature1,feature2,label: 数据处理后的输入特征数据，标签数据  
    :return: new_features,label:特征选择后的特征数据、标签数据  
    """  
    new_features= None  
    # ----- 实现特征选择部分代码 -----  
    # 统计特征值和label的皮尔孙相关系数 对两类特征分别进行排序筛选特征  
    select_feature_number1 = 25  
    select_feature_number2 = 10  
    select_feature1 = SelectKBest(lambda x, Y:  
tuple(map(tuple,np.array(list(map(lambda x:pearsonr(x, Y), x.T))))),  
k=select_feature_number1
```

```

        ).fit(feature1,
np.array(label).flatten()).get_support(indices=True)

    select_feature2 = SelectKBest(lambda X, Y:
tuple(map(tuple,np.array(list(map(lambda x:pearsonr(x, Y), X.T))).T)),
                                k=select_feature_number2
                                ).fit(feature2,
np.array(label).flatten()).get_support(indices=True)

    # 查看排序后特征
    print("select feature1 name:", select_feature1)
    print("select feature2 name:", select_feature2)

    #相关系数筛选出的特征
    s1_feature1=feature1[feature1.columns.values[select_feature1]]
    s1_feature2=feature2[feature2.columns.values[select_feature2]]

    # ----- PCA降维部分代码 -----
    # 选择降维维度
    pca1 = PCA(n_components=2)
    pca2 = PCA(n_components=1)
    feature_pca1 = pca1.fit_transform(s1_feature1)
    feature_pca2 = pca2.fit_transform(s1_feature2)

    # 获取每个主成分的方差变化信息
    variance_explained1 = pca1.explained_variance_ratio_
    variance_explained2 = pca2.explained_variance_ratio_
    print("主成分1的方差变化信息: ", variance_explained1)
    print("主成分2的方差变化信息: ", variance_explained2)
    from mpl_toolkits.mplot3d import Axes3D

    # 可视化标签中不能出现负值
    pca_label = np.array(label).flatten()
    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')
    ax.scatter(feature_pca1[:, 0], feature_pca1[:, 1], feature_pca2[:,
0],c=pca_label)
    plt.show()
    # 对 feature1 和 feature2 进行整合
    feature_pca1_df = pd.DataFrame(feature_pca1)
    feature_pca2_df = pd.DataFrame(feature_pca2)

    new_features = pd.concat([feature_pca1_df, feature_pca2_df], axis=1)
    # -----
    # 返回筛选后的数据
    return new_features, label

def data_split(features, labels):
    """
    数据切分
    :param features, label: 特征选择后的输入特征数据、类标数据
    :return: X_train, X_val, X_test, y_train, y_val, y_test: 数据切分后的训练数据、验证
    数据、测试数据
    """

    X_train, X_val, X_test, y_train, y_val, y_test=None, None, None, None, None,
None

```

```

# ----- 实现数据切分部分代码 -----
# 将 features 和 label 数据切分成训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(features, label,
test_size=0.2, random_state=10, stratify=label)

# 将 X_train 和 y_train 进一步切分为训练集和验证集
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train,
test_size=0.2, random_state=5, stratify=y_train)

# -----

return X_train, X_val, X_test, y_train, y_val, y_test

def plot_learning_curve(estimator, X, y, cv=None, n_jobs=1):
    """
    绘制学习曲线
    :param estimator: 训练好的模型
    :param X: 绘制图像的 X 轴数据
    :param y: 绘制图像的 y 轴数据
    :param cv: 交叉验证
    :param n_jobs:
    :return:
    """
    train_sizes, train_scores, test_scores = learning_curve(estimator, X, y,
cv=cv, n_jobs=n_jobs)
    train_scores_mean = np.mean(train_scores, axis=1)
    test_scores_mean = np.mean(test_scores, axis=1)

    plt.figure('Learning Curve', facecolor='lightgray')
    plt.title('Learning Curve')
    plt.xlabel('train size')
    plt.ylabel('score')
    plt.grid(linestyle=":")
    plt.plot(train_sizes, train_scores_mean, label='training score')
    plt.plot(train_sizes, test_scores_mean, label='val score')
    plt.legend()
    plt.show()

def search_model(X_train, y_train, X_val, y_val, model_save_path):
    """
    创建、训练、优化和保存深度学习模型
    :param X_train, y_train: 训练集数据
    :param X_val, y_val: 验证集数据
    :param save_model_path: 保存模型的路径和名称
    :return:
    """
    # ----- 实现模型创建、训练、优化和保存等部分的代码 -----
    -----
    # 创建监督学习模型 以决策树为例
    clf = tree.DecisionTreeClassifier(random_state=42)

    # 创建调节的参数列表
    parameters = {'max_depth': range(3, 10),
                  'min_samples_split': range(5, 10)}

    # 创建一个fbeta_score打分对象 以F-score为例
    scorer = make_scorer(fbeta_score, beta=1)

```

```

# 在分类器上使用网格搜索，使用'scorer'作为评价函数
kfold = KFold(n_splits=10) #切割成十份

# 同时传入交叉验证函数
grid_obj = GridSearchCV(clf, parameters, scorer, cv=kfold)

#绘制学习曲线
plot_learning_curve(clf, X_train, y_train, cv=kfold, n_jobs=4)

# 用训练数据拟合网格搜索对象并找到最佳参数
grid_obj.fit(X_train, y_train)

# 得到estimator并保存
best_clf = grid_obj.best_estimator_
joblib.dump(best_clf, model_save_path)

# 使用没有调优的模型做预测
predictions = (clf.fit(X_train, y_train)).predict(X_val)
best_predictions = best_clf.predict(X_val)

# 调优后的模型
print ("best_clf\n-----")
print (best_clf)

# 汇报调参前和调参后的分数
print("\nUnoptimized model\n-----")
print("Accuracy score on validation data:
{:.4f}".format(accuracy_score(y_val, predictions)))
print("Recall score on validation data: {:.4f}".format(recall_score(y_val,
predictions)))
print("F-score on validation data: {:.4f}".format(fbeta_score(y_val,
predictions, beta = 1)))
print("\nOptimized Model\n-----")
print("Final accuracy score on the validation data:
{:.4f}".format(accuracy_score(y_val, best_predictions)))
print("Recall score on validation data: {:.4f}".format(recall_score(y_val,
best_predictions)))
print("Final F-score on the validation data:
{:.4f}".format(fbeta_score(y_val, best_predictions, beta = 1)))

# 保存模型（请写好保存模型的路径及名称）
# -----

def load_and_model_prediction(X_test,y_test,save_model_path):
    """
    加载模型和评估模型
    可以实现，比如：模型优化过程中的参数选择，测试集数据的准确率、召回率、F-score 等评价指标！
    主要步骤：
        1.加载模型（请填写你训练好的最佳模型），
        2.对自己训练的模型进行评估

    :param X_test,y_test: 测试集数据
    :param save_model_path: 加载模型的路径和名称,请填写你认为最好的模型
    :return:
    """

# ----- 实现模型加载和评估等部分的代码 -----
#加载模型

```

```

my_model=joblib.load(save_model_path)

#对测试数据进行预测
copy_test = [value for value in X_test]
copy_predicts = my_model.predict(X_test)

print ("Accuracy on test data: {:.4f}".format(accuracy_score(y_test,
copy_predicts)))
print ("Recall on test data: {:.4f}".format(recall_score(y_test,
copy_predicts)))
print ("F-score on test data: {:.4f}".format(fbeta_score(y_test,
copy_predicts, beta = 1)))
# -----
-

def main():
    """
    监督学习模型训练流程，包含数据处理、特征选择、训练优化模型、模型保存、评价模型等。
    如果对训练出来的模型不满意，你可以通过修改数据处理方法、特征选择方法、调整模型类型和参数等方法重新训练模型，直至训练出你满意的模型。
    如果你对自己训练出来的模型非常满意，则可以进行测试提交！
    :return:
    """
    data_path = "DataSet.xlsx" # 数据集路径

    save_model_path = './results/my_model.m' # 保存模型路径和名称

    # 获取数据 预处理
    feature1,feature2,label = processing_data(data_path)

    #特征选择
    new_features,label = feature_select(feature1, feature2, label)

    #数据划分
    X_train, X_val, X_test,y_train, y_val, y_test =
data_split(new_features,label)

    # 创建、训练和保存模型
    search_model(X_train, y_train,X_val,y_val, save_model_path)

    # 评估模型
    load_and_model_prediction(X_test,y_test,save_model_path)

if __name__ == '__main__':
    main()

```

5.2 模型预测代码(main.py)

```

from mpl_toolkits.mplot3d import Axes3D
import warnings
import itertools
import numpy as np
import pandas as pd

```

```

import seaborn as sns
from time import time
from minepy import MINE
from sklearn import svm
from sklearn import tree
import matplotlib.pyplot as plt
from sklearn import naive_bayes
from scipy.stats import pearsonr
from sklearn.manifold import TSNE
from IPython.display import display
from datetime import datetime as dt
from sklearn.externals import joblib
from sklearn.decomposition import PCA
from sklearn.metrics import fbeta_score
from sklearn.metrics import make_scorer
from sklearn.metrics import recall_score
from sklearn.model_selection import KFold
from sklearn.feature_selection import chi2
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import ShuffleSplit
from sklearn.model_selection import GridSearchCV
from sklearn.feature_selection import SelectKBest
from sklearn.model_selection import learning_curve
from sklearn.model_selection import train_test_split

def data_processing_and_feature_selecting(data_path):
    """
    特征选择
    :param data_path: 数据集路径
    :return: new_features, label: 经过预处理和特征选择后的特征数据、标签数据
    """
    new_features, label = None, None
    # ----- 实现数据处理和特征选择部分代码 -----
    -----
    #导入医疗数据
    data_xls = pd.ExcelFile(data_path)
    data={}

    #查看数据名称与大小
    for name in data_xls.sheet_names:
        df = data_xls.parse(sheet_name=name, header=None)
        data[name] = df

    #获取 特征1 特征2 类标
    feature1_raw = data['Feature1']
    feature2_raw = data['Feature2']
    label = data['label']

    # 初始化一个 scaler, 并将它施加到特征上
    scaler = MinMaxScaler()
    feature1 = pd.DataFrame(scaler.fit_transform(feature1_raw))
    feature2 = pd.DataFrame(scaler.fit_transform(feature2_raw))
    # -----
    # 统计特征值和label的皮尔孙相关系数 对两类特征分别进行排序筛选特征
    select_feature_number1 = 25
    select_feature_number2 = 10

```

```

select_feature1 = SelectKBest(lambda X, Y:
tuple(map(tuple,np.array(list(map(lambda x:pearsonr(x, Y), X.T))).T)),
                                k=select_feature_number1
                                ).fit(feature1,
np.array(label).flatten()).get_support(indices=True)

select_feature2 = SelectKBest(lambda X, Y:
tuple(map(tuple,np.array(list(map(lambda x:pearsonr(x, Y), X.T))).T)),
                                k=select_feature_number2
                                ).fit(feature2,
np.array(label).flatten()).get_support(indices=True)

#相关系数筛选出的特征
s1_feature1=feature1[feature1.columns.values[select_feature1]]
s1_feature2=feature2[feature2.columns.values[select_feature2]]

# ----- PCA降维部分代码 -----
# 选择降维维度
pca1 = PCA(n_components=2)
pca2 = PCA(n_components=1)
feature_pca1 = pca1.fit_transform(s1_feature1)
feature_pca2 = pca2.fit_transform(s1_feature2)

# 获取每个主成分的方差变化信息
variance_explained1 = pca1.explained_variance_ratio_
variance_explained2 = pca2.explained_variance_ratio_

# 可视化标签中不能出现负值
pca_label = np.array(label).flatten()
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(feature_pca1[:, 0], feature_pca1[:, 1], feature_pca2[:,
0],c=pca_label)
plt.show()
# 对 feature1 和 feature2 进行整合
feature_pca1_df = pd.DataFrame(feature_pca1)
feature_pca2_df = pd.DataFrame(feature_pca2)

new_features = pd.concat([feature_pca1_df, feature_pca2_df], axis=1)
# 返回筛选后的数据
return new_features,label

# ----- 请加载您最满意的模型 -----
# 加载模型(请加载你认为的最佳模型)
# 加载模型,加载请注意 model_path 是相对路径, 与当前文件同级。
# 如果你的模型是在 results 文件夹下的 my_model.m 模型, 则 model_path =
'results/my_model.m'
model_path = 'results/my_model.m'

# 加载模型
model = joblib.load(model_path)

# -----

def predict(new_features):
    """

```

加载模型和模型预测

`:param new_features` : 测试数据, 是 `data_processing_and_feature_selecting` 函数的返回值之一。

`:return y_predict` : 预测结果是标签值。

"""

`# ----- 实现模型预测部分的代码 -----`

`# 获取输入图片的类别`

`y_predict = model.predict(new_features)`

`# -----`

`# 返回图片的类别`

`return y_predict`